

Sampling in Particle-Filter Localization and SLAM

Mohammed
GSR/5161/16

September 2026

Summary

Localization and SLAM are estimation problems on a graphical model whose structure is fixed in advance by how the robot moves and what its sensor reads, so the graph itself never has to be learned. What the problem does leave open is the inference: the belief over the robot's poses cannot be computed exactly for this model, so it is approximated by sampling. The noise of the motion and measurement models is almost always set by hand; in this project I estimate it from the data the robot recorded.

I built a testbed that makes a particle filter's sampling choices interchangeable, so that one can be varied while the world, the trajectory and the random seed stay fixed, and measured two of them over twenty simulated worlds in localization and in SLAM: which resampling scheme is used, and when resampling happens. Whether the filter resamples at all is what matters; among the rules that do resample, and among the four schemes, accuracy is the same. The schemes differ instead in how many distinct particles survive, and that reaches the estimate only in FastSLAM with few particles, where a lost particle takes its map with it.

The weights the filter already computes also score how well a given noise setting explains the recording, so the noise can be estimated rather than assumed. The two measurement parameters come back close to the values the world used, 0.19 m against 0.2 m and 0.047 rad against 0.05 rad. The two motion parameters leave no trace in a 30-step run and are recovered on a 400-step one. Learning them at all needs the filter to be tracking, and enough particles for the score to be reliable, so how good the inference is sets a ceiling on what can be learned.

1 Background

A robot that must know where it is has two unreliable sources of information: the controls it was given, which are corrupted by wheel slip, and its measurements of the surroundings, which are corrupted by sensor noise. Combining them into an estimate of the pose, with an honest account of its uncertainty, is probabilistic localization; doing it while building the map is SLAM [1].

The problem as a graphical model

Both are naturally written as a dynamic Bayesian network [2, ch. 6 and 15]. The pose $\mathbf{x}_t$ depends on the previous pose and the control $\mathbf{u}_t$ through the motion model, and each observation $\mathbf{z}_t$ is conditioned on the pose it was taken from and on the map $\mathbf{m}$ through the measurement model (Fig. 1), so

$$p(\mathbf{x}_{0:T}, \mathbf{m}, \mathbf{z}_{1:T} \mid \mathbf{u}_{1:T}) = p(\mathbf{x}_0) p(\mathbf{m}) \prod_{t=1}^T p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t) p(\mathbf{z}_t \mid \mathbf{x}_t, \mathbf{m}). \quad (1)$$

Controls and observations are observed; the trajectory is latent, and in SLAM the map as well. Correspondences are known, so each observation carries the identity of the landmark it measures.

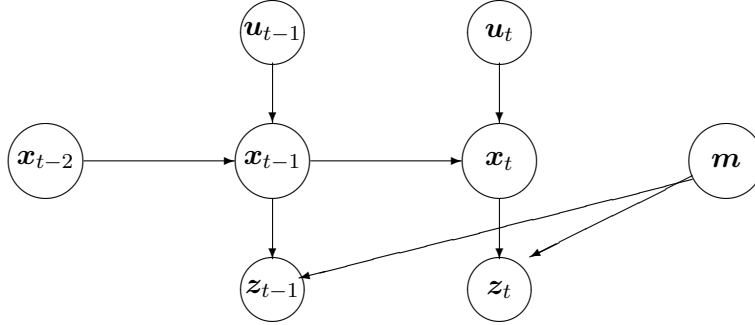


Figure 1: Two time slices of the network. Controls $\mathbf{u}$ and observations $\mathbf{z}$ are observed; poses $\mathbf{x}$ and the map $\mathbf{m}$ are latent. In localization the map is known.

What the filter computes is the *belief*, the distribution over the current pose given everything observed so far. Because $\mathbf{x}_t$ separates past from future it satisfies the Bayes filter recursion, a prediction through the motion model followed by a correction with the new observation [1]:

$$p(\mathbf{x}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \propto p(\mathbf{z}_t \mid \mathbf{x}_t) \int p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t) p(\mathbf{x}_{t-1} \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t-1}) d\mathbf{x}_{t-1}. \quad (2)$$

The state is continuous, so that prediction is an integral. It is tractable in two cases: a discrete state space, where it is a finite sum, and a linear model with Gaussian noise, where the belief stays Gaussian and the recursion is the Kalman filter. Range and bearing measurements are nonlinear in the pose, and under global uncertainty the belief is multimodal, so neither applies and no closed form exists.

The particle filter

Where the integral cannot be carried out it is replaced by samples. The distribution the samples are drawn from is the *proposal*; drawing a pose from the motion model and weighting it by $p(\mathbf{z}_t \mid \mathbf{x}_t)$ is likelihood weighting applied to one time slice, with the prior as the proposal [2, ch. 12], and carrying the weighted set forward makes it sequential [3], [4].

One ingredient has to be added. Over time the weights become products of more and more terms, so they concentrate on a single particle and the set stops representing the belief; how far this has gone is measured by the effective sample size $\text{ESS} = 1 / \sum_i (w^{(i)})^2$. Resampling, which replaces the set by N draws from itself, prevents it, and has no counterpart in the exact algorithm. It costs something in return: drawing with replacement duplicates the heaviest particles and discards the rest, so the number of distinct particles falls. Algorithm 1 shows where each decision sits. Lines 5 and 6 are the subject of this report.

Rao-Blackwellization and FastSLAM

In SLAM the state is the pose together with the map, so the state space gains two dimensions per landmark and sampling it directly is intractable. The network offers a way out. An observation of landmark $\mathbf{m}_k$ has exactly two parents, the pose it was taken from and $\mathbf{m}_k$ itself, so any path between two landmarks must pass through an observation's pose with both edges pointing away from it; a known trajectory blocks every such path. The landmarks are d-separated given the trajectory [2], and the posterior splits:

$$p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) = p(\mathbf{x}_{1:t} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \prod_{k=1}^K p(\mathbf{m}_k \mid \mathbf{x}_{1:t}, \mathbf{z}_{1:t}). \quad (3)$$

Algorithm 1 One step of the particle filter. Lines 5 and 6 are what this report varies.

Require: particles $\{\mathbf{x}_{t-1}^{(i)}, w_{t-1}^{(i)}\}_{i=1}^N$, control $\mathbf{u}_t$, observation $\mathbf{z}_t$

- 1: **for** $i = 1 \dots N$ **do**
- 2: $\mathbf{x}_t^{(i)} \sim q(\mathbf{x} \mid \mathbf{x}_{t-1}^{(i)}, \mathbf{u}_t, \mathbf{z}_t)$ $\triangleright$ *proposal*; the motion model throughout this report
- 3: $\tilde{w}_t^{(i)} \leftarrow w_{t-1}^{(i)} p(\mathbf{z}_t \mid \mathbf{x}_t^{(i)})$ $\triangleright$ weight by the new observation
- 4: **end for**
- 5: $w_t^{(i)} \leftarrow \tilde{w}_t^{(i)} / \sum_j \tilde{w}_t^{(j)}$
- 6: **if** $\text{TRIGGER}(w_t)$ **then** $\triangleright$ *when to resample*
- 7: $\{a_i\} \leftarrow \text{RESAMPLE}(w_t)$ $\triangleright$ *which scheme*: N indices drawn from w_t
- 8: $\mathbf{x}_t^{(i)} \leftarrow \mathbf{x}_t^{(a_i)}, \quad w_t^{(i)} \leftarrow 1/N$
- 9: **end if**
- 10: **return** $\{\mathbf{x}_t^{(i)}, w_t^{(i)}\}$

Sampling only the first factor and solving the rest exactly is Rao-Blackwellization [5], and FastSLAM applies it by giving every particle one trajectory and one extended Kalman filter per landmark [6]. Only the trajectory is sampled, which is what makes particle-filter SLAM affordable.

Learning the parameters

The structure of (1) fixes which distributions appear, not the numbers inside them; write those as θ . Estimating them from data is *parameter* learning rather than structure learning, and here it is parameter learning *with incomplete data* [2, ch. 19], because the trajectory is latent. The network is also a template model, so the same conditional distribution is used at every slice and its parameters are tied across slices [2, ch. 6]: every step is evidence about the same θ , and a longer trajectory is more data rather than more unknowns.

The estimate is the θ that makes the observations most probable, $\hat{\theta} = \arg \max_{\theta} \log p(\mathbf{z}_{1:T} \mid \mathbf{u}_{1:T}, \theta)$, and that maximum is in the right place: the average log likelihood tends to a term free of θ minus the Kullback-Leibler divergence between the true distribution and the model, which vanishes only where the two agree [2, ch. 17]. The likelihood factorizes into one-step terms, each an integral with no closed form, but line 3 of Algorithm 1 already computes what is needed:

$$p(\mathbf{z}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}, \theta) \approx \sum_{i=1}^N \tilde{w}_t^{(i)}, \quad (4)$$

the sum the filter forms in order to normalize [7]. Two consequences follow. This estimate of the likelihood is unbiased, so the estimate of its logarithm is biased downwards, and the bias grows as the particle approximation degrades. And the curvature of the expected score is proportional to the number of steps, so a parameter the observations say little about leaves a flat likelihood, and a longer trajectory sharpens it.

2 Problem statement and objectives

The structure of the model is fixed in advance by the robot’s kinematics and its sensor geometry, so there is no structure to learn. What is left is the inference: the belief cannot be computed in closed form, so it is approximated by a set of samples, and the estimate is only as good as that set. How that set is proposed, weighted and resampled is the first thing measured here.

The motion model and the measurement model each carry a noise level, and in practice these are almost always set by hand. In this project I estimate them instead, from the data the robot itself produced. That is the second thing measured here, and the weights the filter already computes are what make it possible.

Sampling. A filter must choose a proposal distribution, a rule for when to resample, a resampling scheme and a number of particles, and these are usually inherited from whatever implementation is at hand. I built a testbed in which they are interchangeable parts of one filter loop, so that one can be varied while the world, the trajectory and the random seed stay fixed. The filters themselves are existing open-source implementations: the localization filters and resampling schemes of Elfring et al. [8], and FastSLAM 1.0 from PythonRobotics [9].

Two of the four are the subject of this report, lines 6 and 5 of Algorithm 1. The other two are held fixed so that they cannot confound the comparison: the proposal is always the motion model, which draws from the prior, and the number of particles is fixed within each comparison.

The resampling scheme decides how the N new indices are drawn from the weights. All four are unbiased, in that a particle of weight $w^{(i)}$ is expected to be copied $Nw^{(i)}$ times; they differ in how much that count varies around its expectation, and therefore in how many distinct particles survive [10], [11]. Each is drawn below taking $N = 6$ draws from the same six weights, in the colour it keeps in every result figure; a particle is copied once for each tick that lands on it.

Multinomial takes N independent draws from the weights, so the number of copies of a particle is binomial. This has the largest variance of the four: the draws clump, and particles are missed.



Residual first takes $\lfloor Nw^{(i)} \rfloor$ copies of each particle outright (solid), then fills the remaining places multinomially from the leftover weight (dashed). Removing the whole part removes most of the randomness.



Stratified cuts $[0, 1]$ into N equal strata (dotted) and draws one uniform number inside each, forcing the draws to spread over the cumulative weights.



Systematic draws a single uniform $u \in [0, 1/N]$ and takes the N positions $u + (j - 1)/N$. One random number gives the lowest variance of the four, at the cost of correlating the draws.



When to resample decides whether line 6 runs at all on a given step. Resampling fights degeneracy but discards particles, so a rule has to balance the two.

- *Every step*, and *never*, as the two extremes.
- *An ESS threshold*: resample when the effective sample size falls below a fraction of N , tested at 0.2, 0.5 and 0.8. This measures degeneracy directly.
- *A largest-weight threshold*: resample when one particle's weight rises above 0.1, 0.2 or 0.5. This reacts to a single particle taking over, which the ESS notices more slowly.

Parameter learning. The noise values are parameters of the network’s two conditional distributions, and the weights the filter already computes estimate the marginal likelihood of the observations under them, so they can be estimated from a recorded run instead of assumed. This asks not how well the filter infers under a given model, but which parts of the model the data can determine at all.

3 Experiments

Method

The four sampling choices are interchangeable components of a single filter loop, each selected by name, so a configuration is a set of names and a particle count. The same loop serves both problems; what differs is the state a particle carries, a pose in localization and a pose with its own map in SLAM.

Two simulated worlds are used, both taken unchanged from the reference implementations. The *localization world* is a 10 m square, cyclic at its edges, with four landmarks. The robot takes 30 steps of 0.25 m and 0.02 rad, with motion noise of 0.005 m and 0.002 rad, and measures range and bearing to all four landmarks with noise of 0.2 m and 0.05 rad. Particles start uniformly over the world and all headings, so the opening steps are global localization; the learning experiments add a setting in which they start around the true initial pose, so the filter tracks instead. The *SLAM world* is the FastSLAM scenario of its reference implementation: 500 steps among eight landmarks, observations within 20 m with noise of 0.3 m and 2° , and controls corrupted by noise and a constant yaw-rate offset.

A run is one configuration on one seed, and the seed fixes the world, the trajectory and the filter’s randomness. The same seeds are used for every variant in a comparison, so two variants differ only in the choice being varied. Every configuration is run on twenty seeds, and every table reports the mean and standard deviation over them. A difference is called clear when the 95% confidence intervals of the two means do not overlap, and it is only taken between variants sharing the value of the other varied setting.

What is measured: accuracy, as position RMSE and as ATE after the best rigid alignment, plus map error in SLAM; degeneracy, as the mean effective sample size and the fraction of distinct particles surviving a resampling step; consistency, as the median NEES, about 3 when the particle spread matches the error; cost, as runtime per step; and fit, as the log likelihood (4) of the observations. Runtime compares implementations rather than schemes: three of the four reused resamplers search the cumulative weights in a way that costs $O(N^2)$, where all four can be written in $O(N)$, so they differ by an order of magnitude for reasons that have nothing to do with the sampling.

The filters are the reference implementations, adapted where the experiments require it: the sampling steps are made swappable, the models are vectorized over particles and moved to log weights, and the code is checked against the originals. The adapted code is in the repository.

Sampling

Two experiments, each varying one sampling choice alone. The proposal is the motion model throughout and the number of particles is fixed within each comparison, so neither can confound the result.

Resampling scheme

The four schemes at 50, 200 and 1000 particles in localization, resampling at every step so that only the scheme differs (240 runs), and the same four in FastSLAM 1.0 at 10 and 50 particles (160 runs).

In localization the four are equally accurate: at every particle count their errors differ by less than the seed-to-seed spread. They differ clearly in how many distinct particles survive a resampling step, in the order their variance predicts: with 1000 particles systematic keeps 72% distinct, stratified 67%, residual 65% and multinomial 54%, and the same order holds at 50 and 200.

That difference reaches the estimate in one place. In FastSLAM a particle that is lost takes its map with it, so with 10 particles systematic gives an ATE of 0.23m against 0.32m for multinomial, and a map error of 0.105m against 0.139m.

Does the resampling scheme matter? (localization)

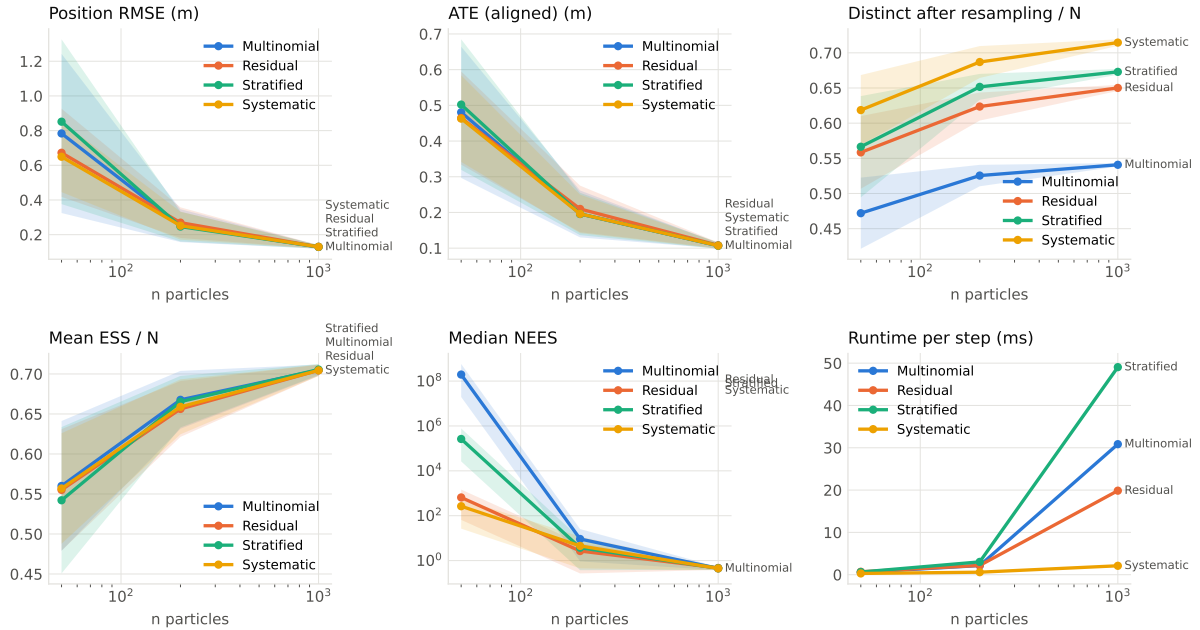


Figure 2: Resampling schemes in localization. The accuracy panels lie on top of one another; the distinct-particle panel separates them.

Table 1: Resampling schemes in FastSLAM 1.0, where each particle carries its own map. Mean $\pm$ standard deviation over seeds.

Resampler	Particles	Seeds	Position RMSE (m)	ATE (aligned) (m)	Map error (aligned) (m)	Distinct after resampling / N	Mean ESS / N	Runtime per step (ms)
Multinomial	10	20	0.96 $\pm$ 0.55	0.324 $\pm$ 0.064	0.139 $\pm$ 0.04	0.624 $\pm$ 0.0043	0.893 $\pm$ 0.0051	1.5 $\pm$ 0.012
Multinomial	50	20	0.975 $\pm$ 0.47	0.223 $\pm$ 0.037	0.0965 $\pm$ 0.024	0.61 $\pm$ 0.002	0.893 $\pm$ 0.0025	2.71 $\pm$ 0.01
Residual	10	20	1.02 $\pm$ 0.84	0.277 $\pm$ 0.048	0.13 $\pm$ 0.037	0.785 $\pm$ 0.0024	0.892 $\pm$ 0.0041	1.57 $\pm$ 0.018
Residual	50	20	0.759 $\pm$ 0.41	0.209 $\pm$ 0.055	0.0832 $\pm$ 0.016	0.778 $\pm$ 0.0025	0.892 $\pm$ 0.0028	2.89 $\pm$ 0.02
Stratified	10	20	1.06 $\pm$ 0.71	0.247 $\pm$ 0.058	0.114 $\pm$ 0.037	0.801 $\pm$ 0.0042	0.893 $\pm$ 0.0043	1.51 $\pm$ 0.0073
Stratified	50	20	0.58 $\pm$ 0.31	0.205 $\pm$ 0.039	0.069 $\pm$ 0.02	0.787 $\pm$ 0.0016	0.892 $\pm$ 0.0021	2.8 $\pm$ 0.0088
Systematic	10	20	0.973 $\pm$ 0.61	0.233 $\pm$ 0.044	0.105 $\pm$ 0.032	0.86 $\pm$ 0.0038	0.895 $\pm$ 0.0037	1.46 $\pm$ 0.023
Systematic	50	20	0.879 $\pm$ 0.54	0.242 $\pm$ 0.1	0.0754 $\pm$ 0.016	0.858 $\pm$ 0.0019	0.892 $\pm$ 0.0024	2.35 $\pm$ 0.17

When to resample

Eight rules in localization with 100 particles and systematic resampling: at every step, never, an ESS threshold of 0.2, 0.5 or 0.8, and a largest-weight threshold of 0.1, 0.2 or 0.5 (160 runs).

Whether the filter resamples is what matters. Not resampling triples the error, 1.39m against roughly 0.46m, and leaves the effective sample size at 1% of N : with 100 particles a single one carries the belief, and the filter is confidently wrong rather than merely inaccurate. Every rule that does resample gives the same error, between 0.45m and 0.49m.

What the threshold changes is cost against impoverishment. It sets how often resampling happens, between a third of the steps and all of them, and resampling less often leaves each step with more degenerate weights, so fewer particles survive the redraw: 25% at a third of the steps against 64% at every step.

When should the filter resample?

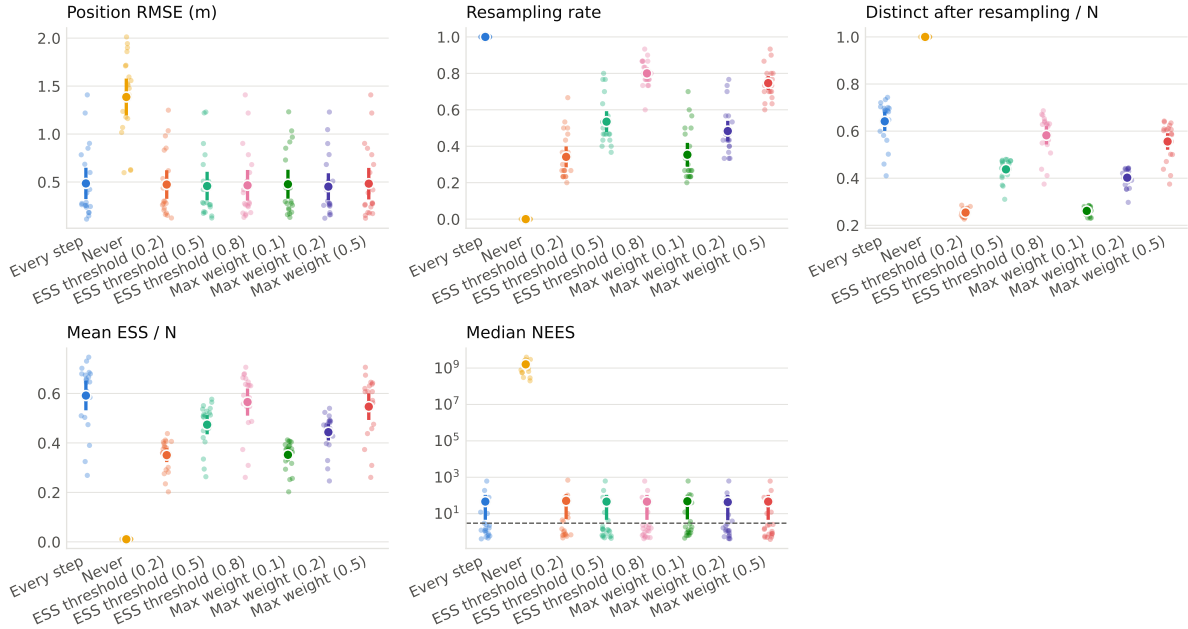


Figure 3: When to resample, in localization with 100 particles. Never resampling stands apart; the rules that do resample are indistinguishable.

Learning the noise

Each of the four noise parameters is swept in turn while the other three are held at the values the world uses, and every value is scored by the log likelihood (4) of the recorded measurements. The filter uses the motion-model proposal, systematic resampling, an ESS threshold of 0.5 and 2000 particles, and starts around the true initial pose (640 runs).

Is learning better than setting the noise by hand?

For the measurement noise, yes. Table 2 puts the three values side by side: the hand-set range noise is twice what the world uses and the hand-set bearing noise six times it, while the learned values are within 5% of the truth. The filter is more accurate at the learned setting, 0.063 m against 0.071 m, about 11% better. The gain is modest because a filter is forgiving of noise set too wide.

For the motion noise, no. Its sweeps are flat, the log likelihood holding at 205 to 206 while the assumed noise varies by a factor of 30, so there is no learned value to put in place of the hand-set one. The true values are small next to the measurement noise the filter corrects with at every step, so the observations barely depend on them.

Which estimates to trust can be decided without knowing the truth, by asking whether separate recordings agree. Each of the twenty seeds is an independent recording, and the last column of Table 2 counts how many land on the same value: all twenty for the bearing noise, seventeen for the range noise, while the motion estimates scatter across the whole sweep. A parameter the data pins down gives the same answer from every recording; one it does not gives a different answer each time.

Table 2: What was learned, against the value set by hand in the upstream code and the value the world used. The last column counts how many of the twenty recordings pick the same value on their own.

Parameter	Set by hand	Learned	World	Recordings agreeing
Measurement, range	0.4 m	0.19 m	0.2 m	17 of 20
Measurement, bearing	0.3 rad	0.047 rad	0.05 rad	20 of 20
Motion, forward	0.1 m	not identified	0.005 m	3 of 20
Motion, turn	0.2 rad	not identified	0.002 rad	0 of 20

Can the measurement noise be learned from the data? (range)

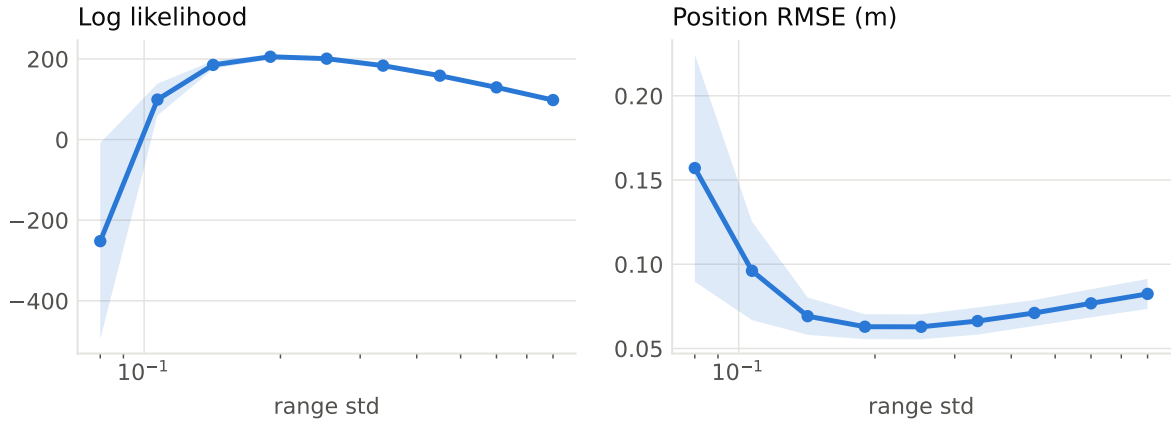


Figure 4: The assumed range noise: the log likelihood peaks at the value the world uses, 0.2 m, and the error is lowest at the same place.

The shape of the peak is the one the model predicts. Too little assumed noise makes the measurements that arrived look impossible and the score falls steeply, from 205 at 0.19 m to -252 at 0.08 m; too much makes every pose explain them equally well and the score decays slowly, to 98 at 0.8 m. For a Gaussian residual of true width σ^* scored under σ the expected score is $-\log \sigma - \sigma^{*2}/2\sigma^2$ up to a constant, maximized at σ^* but falling like σ^{-2} below it and only like $\log \sigma$ above. With 120 range residuals that expression predicts the whole measured curve with nothing fitted: at 0.142, 0.253, 0.45 and 0.8 m it gives -18 , -6 , -49 and -110 relative to the peak, against measured -20 , -5 , -47 and -108 . It parts company only at the far left, predicting -205 where -457 is measured, because there the weights degenerate and the estimator's downward bias takes over.

What learning requires

Three further sets of runs, each changing one condition: the turn sweep on a 400-step trajectory instead of 30 (140 runs); the range sweep from the uniform start as well as from a known pose (360 runs); and the range sweep at 25 to 2000 particles, and under each resampling scheme at 200 particles (1440 runs).

Learning needs enough data, a filter that is already tracking, and enough particles. The resampling scheme makes no difference.

On 400 steps the flat turn curve gains a maximum at the true value, and an assumed drift ten times too small is rejected by thousands of nats; the twenty recordings concentrate instead of scattering. Identifiability is a property of the data, not of the method: because the parameters are tied across time slices, the curvature of the likelihood grows with the number of steps, as Section 1 predicts.

From the uniform start the maximum disappears altogether. With the true, small noise the filter never finds the robot within 30 steps: the position error is 2.3m against 0.063m from a known start, and the log likelihood is four orders of magnitude lower. The score then simply rises with wider assumed noise, because wider noise lets the particles spread enough to be useful, and only 2 of 20 recordings agree. What the likelihood reports in this regime is which setting lets this filter work, not which setting generated the data.

At the true value the estimated log likelihood rises from 28 ± 316 with 25 particles to 205 ± 13 with 2000, and agreement rises with it, from no two recordings agreeing to 17 of 20: both the bias and the spread shrink as the approximation improves. Under the four resampling schemes at 200 particles the answer is the same throughout, 0.19 m, with 14 or 15 of 20 recordings agreeing.

Does more data make the motion noise identifiable? (turn, 400 steps)

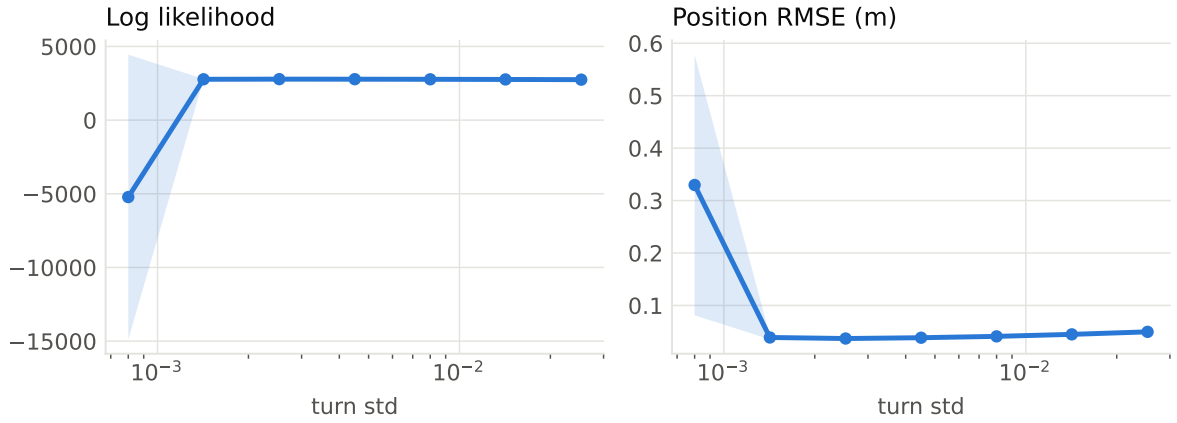


Figure 5: The turn sweep over 400 steps: enough steps identify a value that 30 steps cannot.

Does learning depend on the filter tracking at all?

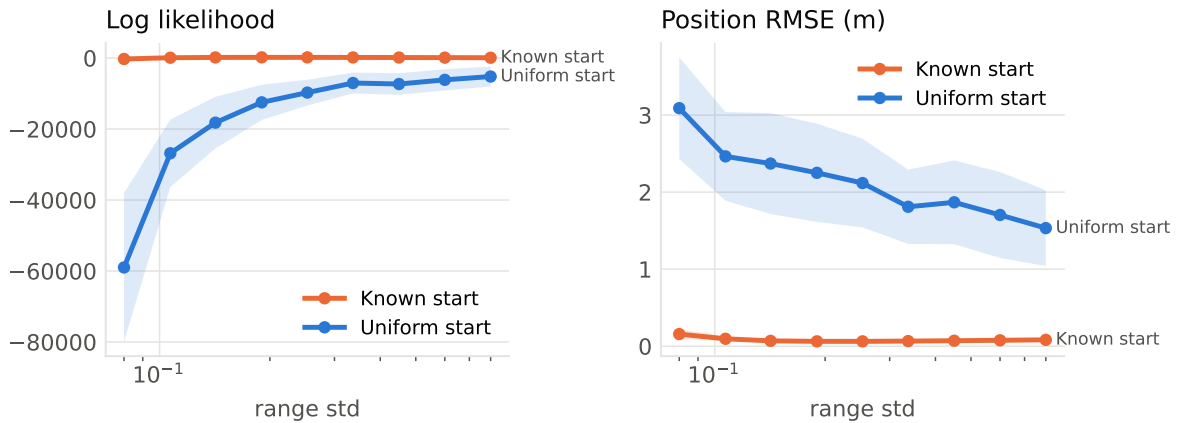


Figure 6: The range sweep from a known start and from the uniform start. Note the scale: from the uniform start the estimate is four orders of magnitude lower and has no maximum.

4 Discussion and conclusion

Which choices mattered. The number of particles and the decision to resample at all decided most of the accuracy. The choices made inside resampling decided particle diversity instead, and

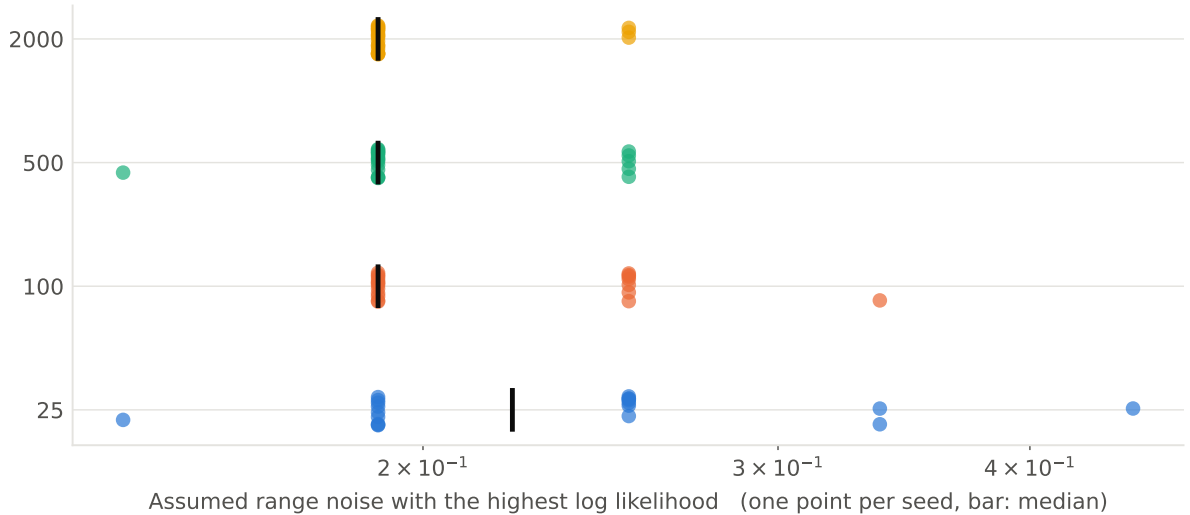


Figure 7: The range noise each recording picks out on its own, against the number of particles.

Table 3: The range noise picked out under each resampling scheme, with 200 particles.

Resampler	Seeds	Best assumed range noise (median)	Range over seeds	Seeds at the median
Multinomial	20	0.19	0.19 to 0.45	15 of 20
Residual	20	0.19	0.19 to 0.337	14 of 20
Stratified	20	0.19	0.19 to 0.253	15 of 20
Systematic	20	0.19	0.19 to 0.337	14 of 20

the two are not the same thing: a difference in diversity reaches the estimate only when particles are scarce and each one carries a map. The proposal sits between them, mattering most where particles are few and losing its advantage as they are added. A measurement-informed proposal also costs more per particle, so the fair comparison is at equal runtime rather than at equal particle count, and there its advantage is much smaller. Adaptive resampling buys no accuracy in these worlds but saves resampling steps at no cost.

What limits learning. Two limits, of different kinds. The inference is one: the objective is estimated by the same weights that do the filtering, so a filter that has not found the robot reports which setting lets it work rather than which setting generated the data, and one with too few particles reports noise. Inference quality is a precondition for learning, not something learning improves. The data is the other: the motion parameters are well defined and simply undetermined by 30 steps of this world, which no amount of computation repairs. Agreement between independent recordings separates the two cases cheaply. The one-sided shape of the likelihood also explains why noise set by hand is rationally kept large, as the inherited settings are.

Limitations. The worlds are small and simulated, with known correspondences; real sensors, unknown data association and larger maps would make degeneracy and proposal quality more important. Each experiment varies one choice while the others are fixed, so interactions are not measured. The rules for when to resample were compared only in localization, and the resampling schemes in SLAM only with FastSLAM 1.0. The proposal and the number of particles were held fixed rather than varied, so nothing here says how the two choices studied interact with them. The learning experiments estimate one parameter at a time from a known start,

so they do not give the joint maximum over all four, and a sweep is a cruder optimizer than expectation maximization.

Conclusion. Localization and SLAM, written as a dynamic Bayesian network, make the same three demands as any graphical model: a structure that justifies a factorization, an inference procedure for what the structure leaves open, and parameters that have to come from somewhere. This project took the structure as given and worked on the other two. On the inference side it measured which sampling choices change the answer, and found that most of the accuracy is decided before the resampling scheme is chosen. On the learning side it estimated the model’s own noise parameters from recorded runs, recovered the measurement noise to within one step of the sweep, and showed that the motion noise is identifiable only with a longer trajectory. The two sides meet in one result: the quality of the approximate inference sets the ceiling on what can be learned.

With more time the obvious next steps would be MCMC moves after resampling [12], which attack particle impoverishment directly; the rules for when to resample in SLAM; estimating the four parameters together by expectation maximization instead of one sweep at a time; and running the whole thing on real data, where the correspondences are not given.

References

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005. [Online]. Available: <https://mitpress.mit.edu/9780262201629/probabilistic-robotics/>.
- [2] D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009. [Online]. Available: <https://mitpress.mit.edu/9780262013192/probabilistic-graphical-models/>.
- [3] N. J. Gordon, D. J. Salmond, and A. F. M. Smith, “Novel approach to nonlinear/non-Gaussian Bayesian state estimation,” *IEE Proceedings F (Radar and Signal Processing)*, vol. 140, no. 2, pp. 107–113, 1993. DOI: [10.1049/ip-f-2.1993.0015](https://doi.org/10.1049/ip-f-2.1993.0015).
- [4] M. S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp, “A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking,” *IEEE Transactions on Signal Processing*, vol. 50, no. 2, pp. 174–188, 2002. DOI: [10.1109/78.978374](https://doi.org/10.1109/78.978374).
- [5] A. Doucet, N. de Freitas, K. Murphy, and S. Russell, “Rao-Blackwellised particle filtering for dynamic Bayesian networks,” in *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*, 2000. arXiv: [1301.3853](https://arxiv.org/abs/1301.3853).
- [6] M. Montemerlo, S. Thrun, D. Koller, and B. Wegbreit, “FastSLAM: A factored solution to the simultaneous localization and mapping problem,” in *Proceedings of the AAAI National Conference on Artificial Intelligence*, 2002. [Online]. Available: <https://cdn.aaai.org/AAAI/2002/AAAI02-089.pdf>.
- [7] A. Doucet and A. M. Johansen, “A tutorial on particle filtering and smoothing: Fifteen years later,” in *The Oxford Handbook of Nonlinear Filtering*, Oxford University Press, 2011. [Online]. Available: https://www.stats.ox.ac.uk/~doucet/doucet_johansen_tutorialPF2011.pdf.
- [8] J. Elfring, E. Torta, and R. van de Molengraft, “Particle filters: A hands-on tutorial,” *Sensors*, vol. 21, no. 2, p. 438, 2021. DOI: [10.3390/s21020438](https://doi.org/10.3390/s21020438).
- [9] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, *PythonRobotics: A Python code collection of robotics algorithms*, 2018. arXiv: [1808.10703](https://arxiv.org/abs/1808.10703). [Online]. Available: <https://github.com/AtsushiSakai/PythonRobotics>.

- [10] R. Douc, O. Cappé, and E. Moulines, “Comparison of resampling schemes for particle filtering,” in *Proceedings of the International Symposium on Image and Signal Processing and Analysis (ISPA)*, 2005, pp. 64–69. DOI: [10.1109/ISPA.2005.195385](https://doi.org/10.1109/ISPA.2005.195385).
- [11] J. D. Hol, T. B. Schön, and F. Gustafsson, “On resampling algorithms for particle filters,” in *IEEE Nonlinear Statistical Signal Processing Workshop*, 2006, pp. 79–82. DOI: [10.1109/NSSPW.2006.4378824](https://doi.org/10.1109/NSSPW.2006.4378824).
- [12] W. R. Gilks and C. Berzuini, “Following a moving target—Monte Carlo inference for dynamic Bayesian models,” *Journal of the Royal Statistical Society: Series B*, vol. 63, no. 1, pp. 127–146, 2001. DOI: [10.1111/1467-9868.00280](https://doi.org/10.1111/1467-9868.00280).